CODE-DOC
Campus Reachability and Emergency Route Audit
Using Recursive DFS and BFS in Python
Submitted By
Name: Kanuri Thanush

Roll Number: 25MVCAIR0059

Course: MCA

Language Used: Python 3
Problem Statement
The objective of this project is to develop a graph-based campus auditing system that analyzes the connectivity of different areas within a campus. The campus consists of several locations represented as vertices, while the walkways connecting these locations are represented as undirected edges.

The application performs three major operations:
1. Identify all connected components using Recursive Depth First Search (DFS).
2. Perform DFS and BFS traversals from a given emergency station (source vertex).
3. Find the shortest route from the emergency station to requested destination vertices using the Breadth First Search parent array.
The program must always process neighbouring vertices in increasing numerical order so that every execution produces the same output regardless of the order in which edges are entered.
Objective
The main objectives of this project are:
- To represent a campus as an undirected graph.
- To identify every connected component of the graph.
- To assign every vertex the smallest numbered vertex of its connected component.
- To perform Recursive DFS traversal.
- To perform Queue-based BFS traversal.
- To compute minimum-edge routes using BFS.
- To print unreachable destinations when no path exists.
- To produce deterministic outputs by sorting adjacency lists.
Introduction
Graphs are one of the most important data structures used in computer science. Many real-world problems such as road navigation, computer networks, social networks, transportation systems, and campus maps can be represented using graphs.

In this project, every building or location inside the campus is represented as a vertex, while every walkway connecting two buildings is represented as an edge.

Example:
Academic Block ----- Library
|
|
Computer Lab ----- Cafeteria

Hostel ----- Playground

This graph contains two connected areas.

The first connected area contains:
Academic Block
Library
Computer Lab
Cafeteria

The second connected area contains:
Hostel
Playground

The program automatically detects these connected areas and labels them.
Concepts Used
The project uses the following graph concepts:
1. Graph
A graph consists of vertices and edges.

Example
0 ----- 1
| |
| |
2 ----- 3

Vertices:
0 1 2 3

Edges:
0-1
0-2
1-3
2-3

2. Adjacency List
Instead of storing the graph as a matrix, an adjacency list is used.

Example:
0 -> 1 2

1 -> 0 3

2 -> 0 3

3 -> 1 2

Advantages:
- Less memory usage.
- Faster traversal.
- Easy to store sparse graphs.
3. Connected Component
A connected component is a group of vertices where every vertex can be reached from every other vertex.

Example
0 ----- 1

| |

2 ----- 3

4 ----- 5

Connected Components:

Component 1

0
1
2
3

Component 2
4
5

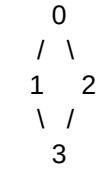
Total Components:
2

4. Depth First Search (DFS)
Depth First Search explores one path completely before backtracking.

Example
0
|
1
|
3
|
2

DFS Order:
0 1 3 2

The project uses Recursive DFS.
5. Breadth First Search (BFS)
Breadth First Search explores level by level.

Example


Traversal
Level 0

0

?

Level 1

1 2

?

Level 2

3

BFS Order
0 1 2 3

6. Parent Array
During BFS, every newly discovered vertex stores the vertex from which it was discovered.

Example
parent[1]=0

parent[2]=0

parent[3]=1

Parent Array
Vertex

0 1 2 3

Parent

-1 0 0 1

7. Distance Array
The distance array stores the minimum number of edges from the source.

Example
Source = 0

Distance Array
Vertex

0 1 2 3

Distance

0 1 1 2

This information is later used to print the shortest routes.
Software Requirements
- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python 3
- IDE: Visual Studio Code (VS Code)
- Python Version: Python 3.10 or above

Part 2: Algorithm and Code Explanation
Algorithm
The program is divided into five major stages.
Step 1: Read the Input
The program first reads:
- Number of vertices (n)
- Number of edges (m)
- Every edge of the graph
- Source vertex (s)
- Number of route queries (q)
- Target vertices
The graph is stored using an adjacency list.

Example
6 5
0 2
0 1

1 3
2 3
4 5

Adjacency List
0 -> 1 2

1 -> 0 3

2 -> 0 3

3 -> 1 2

4 -> 5

5 -> 4

After reading all edges, every adjacency list is sorted.

Sorting guarantees that neighbours are always visited in increasing numerical order.
Step 2: Find Connected Components
The program performs Recursive DFS over the whole graph.

Algorithm:
Mark all vertices as unvisited

For every vertex

    if vertex is not visited

        start Recursive DFS

        label every reachable vertex

        increase component count

Suppose the graph is
0 ----- 1
| |
| |
2 ----- 3

4 ----- 5

DFS starting from
0

visits
0
1
3
2

All these vertices receive label
0

Next DFS starts from
4

visits
4
5

Both receive label
4

Finally
Labels

0 0 0 0 4 4

Number of Components
2

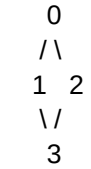
Step 3: Perform DFS Traversal
A second Recursive DFS is performed only from the given source.

Purpose
- Explore only the source component.
- Record visiting order.
Algorithm
Visit source

Record source

Visit every neighbour recursively

Continue until no neighbour remains

Example


Traversal
0

?

1

?

3

?

2

Output
DFS

0 1 3 2

The vertex is recorded immediately after entering it.
Step 4: Perform BFS Traversal
Breadth First Search uses a Queue.

Algorithm
Insert source into queue

Mark source visited

While queue is not empty

Remove front vertex

Record it

Visit every neighbour

Mark neighbour visited

Save parent

Save distance

Insert neighbour

Example
Queue

0

Remove
0

Insert
1
2

Queue
1 2

Remove
1

Insert
3

Queue
2 3

Remove
2

Queue
3

Remove
3

Traversal
0 1 2 3

Step 5: Build Shortest Route
The program never runs another shortest-path algorithm.

Instead it uses the Parent Array produced during BFS.

Example

Parent Array
Vertex

0 1 2 3

Parent

-1 0 0 1

Target
3

Trace backwards
3

?

1

?

0

Reverse
0 1 3

Distance
2

Output
ROUTE 3 2 0 1 3

Overall Algorithm
Read Input

?

Create Adjacency List

?

Sort Every Neighbour List

?

Recursive DFS
Find Connected Components

?

Print Components

?

Recursive DFS
From Source

?

Print DFS Order

?

Queue Based BFS

?

Store Parent Array

?

Store Distance Array

?

Answer Queries

?

Print Routes

Detailed Code Explanation
Import Statements
import sysfrom collections import deque

Purpose
sys

Used to read complete input efficiently.

deque

Used as a queue during BFS.
Reading Input
data = sys.stdin.read().split()

Explanation
Reads every value from standard input.

Example Input
6 5
0 2
0 1

becomes
['6','5','0','2','0','1']

Creating Adjacency List
adj = [[] for _ in range(n)]

Creates one empty list for every vertex.

Example
n = 4

Creates
[[],[],[],[]]

Reading Every Edge
adj[u].append(v)adj[v].append(u)

Because the graph is undirected.

Example

Edge
0 2

becomes
0 -> 2

2 -> 0

Sorting the Adjacency List
for lst in adj: lst.sort()

Example

Before
0

2 1

After
0

1 2

Sorting makes DFS and BFS deterministic.
Connected Component Variables
label = [-1]*nvisited = [False]*ncomponents = 0

Purpose

label

Stores component number.

visited

Checks whether vertex has already been explored.

components

Counts total connected components.
Recursive DFS Function
def label_dfs(u, comp_id):

Parameters
u

Current vertex

comp_id

Smallest vertex of component

Inside function
visited[u]=Truelabel[u]=comp_id

Marks vertex visited and assigns label.
Finding Connected Components
for v in range(n):

Starts DFS from the smallest unvisited vertex.

Example
0

?

DFS

?

4

?

DFS

Produces
2 Components

DFS Traversal
dfs_order=[]

Stores DFS visiting order.

Example

0

?

1

?

3

?

2

Stored as
0 1 3 2

BFS Variables
dist=[]parent=[]queue

Purpose

dist

Stores minimum distance from source.

parent

Stores previous vertex.

queue

Controls BFS order.
Distance Array
Example
Source = 0

Distance
Vertex

0 1 2 3

Distance

0 1 1 2

Parent Array
Example
Vertex

0 1 2 3

Parent

-1 0 0 1

Meaning

3 discovered from 1

1 discovered from 0

Route Reconstruction
Algorithm
Target

?

Follow Parent

?

Reach Source

?

Reverse List

?

Print Path

Example
3

?

1

?

0

Reverse
0 1 3

Output Section
The program prints:
COMPONENTS

?

LABELS

?

DFS

?

BFS

?

ROUTE

Exactly in the format required by the assignment specification.

Time Complexity Analysis
The efficiency of the program depends on the graph traversal algorithms used. Both Depth First Search (DFS) and Breadth First Search (BFS) are efficient graph traversal techniques.
1. Building the Graph
The program reads all the edges and stores them in an adjacency list.
adj[u].append(v)adj[v].append(u)

There are m edges.

Time Complexity
\[
O(m)
\]
where
- n = Number of vertices
- m = Number of edges
2. Sorting the Adjacency List
Every adjacency list is sorted.
for lst in adj: lst.sort()

Sorting guarantees neighbours are visited in increasing order.

Worst-case complexity
\[
O(m\log n)
\]
3. Connected Component DFS
Every vertex is visited exactly once.

Every edge is also visited once.

Time Complexity
\[
O(n+m)
\]
4. DFS Traversal
The DFS traversal starts only from the source vertex.

Every reachable vertex is visited once.

Time Complexity
\[
O(n+m)
\]
5. BFS Traversal
Breadth First Search also visits every reachable vertex only once.

Each edge is examined only once.

Time Complexity
\[
O(n+m)
\]
6. Route Reconstruction

For every query, the program follows the parent array.

If the route contains k vertices

Time Complexity
$$
O(k)
$$
Worst case
$$
O(n)
$$
Overall Time Complexity
The complete program executes in
$$
O(m\log n)+O(n+m)
$$
which is commonly written as
$$
\boxed{O(m\log n+n+m)}
$$
Since the maximum number of vertices is only 200, the program executes very quickly.
Space Complexity
The program stores several arrays.

Data Structure	Space
Adjacency List	O(n+m)
Visited Array	O(n)
Label Array	O(n)
Parent Array	O(n)
Distance Array	O(n)
Queue	O(n)
DFS Order	O(n)
BFS Order	O(n)

Total Space Complexity
$$
\boxed{O(n+m)}
$$
Test Case 1 (Original)
Input
```
5 4
0 1
1 2
2 3
3 4
0 2
4
2
```

Expected Output
```
COMPONENTS 1
LABELS 0 0 0 0 0
DFS 0 1 2 3 4
BFS 0 1 2 3 4
ROUTE 4 4 0 1 2 3 4
ROUTE 2 2 0 1 2
```

Explanation
All five vertices belong to one connected component.

DFS visits every vertex recursively.

BFS finds the shortest route.

The route to vertex 4 contains four edges.
Test Case 2 (Original)
Input
7 4
0 1
1 2
3 4
5 6
0 3
2
5

Expected Output
COMPONENTS 3
LABELS 0 0 0 3 3 5 5
DFS 0 1 2
BFS 0 1 2
ROUTE 3 UNREACHABLE
ROUTE 5 UNREACHABLE

Explanation
The graph contains three connected components.
0-1-2

3-4

5-6

Source vertex is 0.

Vertices 3 and 5 belong to different components.

Therefore they are unreachable.
Test Case 3 (Original)
Input
6 6
0 1
0 2
1 3
2 4
3 5
4 5
0 3
5
4
0

Expected Output
COMPONENTS 1
LABELS 0 0 0 0 0 0
DFS 0 1 3 5 4 2

BFS 0 1 2 3 4 5
ROUTE 5 3 0 1 3 5
ROUTE 4 2 0 2 4
ROUTE 0 0 0

Explanation
Only one connected component exists.

DFS explores one branch completely before backtracking.

BFS explores level by level.

The shortest path to vertex 5 is
0 -> 1 -> 3 -> 5

Sample Test Case 1
Input
6 5
0 2
0 1
1 3
2 3
4 5
0 3
3
5
0

Expected Output
COMPONENTS 2
LABELS 0 0 0 0 4 4
DFS 0 1 3 2
BFS 0 1 2 3
ROUTE 3 2 0 1 3
ROUTE 5 UNREACHABLE
ROUTE 0 0 0

Explanation
The graph has two connected components.
0-1
| |
2-3

4-5

DFS visits
0
?

1
?

3
?

2

BFS visits

0

?

1 2

?

3

Vertex 5 belongs to another connected component.

Therefore it is unreachable.
Sample Test Case 2
Input
3 0
1 2
1
0

Expected Output
COMPONENTS 3
LABELS 0 1 2
DFS 1
BFS 1
ROUTE 1 0 1
ROUTE 0 UNREACHABLE

Explanation
No edges exist.

Every vertex forms its own connected component.

DFS and BFS both visit only the source vertex.
Conclusion
This project successfully demonstrates the implementation of graph traversal algorithms using Recursive Depth First Search (DFS) and Breadth First Search (BFS). The program identifies all connected components in the graph, labels each vertex with the smallest vertex number in its connected component, performs deterministic DFS and BFS traversals, and reconstructs minimum-edge routes using the BFS parent array.

By sorting adjacency lists before traversal, the output remains consistent and reproducible regardless of the order in which edges are provided. The project also highlights the practical use of graphs in representing campus walkways and emergency navigation systems. Overall, the implementation satisfies all the requirements specified in the problem statement while maintaining efficient time and space complexity

Team Members Contribution
Name Roll Number Role Contribution
Kanuri Thanush 25MVCAIR0059
Team Leader Led the project, designed the overall algorithm, implemented the Python program using Recursive DFS and BFS, integrated all modules, tested the application with different test cases, analyzed time and space complexity, debugged the code, and prepared the complete project documentation (CODE-DOC).
Ramagiri Manish 25MVCAIR0069
Team Member Assisted in understanding the problem statement, contributed to testing the program with sample and custom test cases, verified the correctness of outputs, and participated in documentation review.

Challa Bharat 25MVCAIR0020
Team Member Assisted in graph analysis, verified DFS and BFS traversal results, helped identify edge cases, and supported testing and validation of the implementation.
Devata Sai Nihar 25MVCAIR0030
Team Member Contributed to requirement analysis, checked algorithm flow, assisted in preparing test cases and validating the final outputs, and reviewed the documentation before submission.